



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Hoja de Referencia: JavaScript Moderno y Web Components

Esta guía rápida le proporciona los conceptos clave, la sintaxis estándar y ejemplos prácticos necesarios para resolver la lógica de cliente del **Laboratorio 2** utilizando JavaScript nativo.

1. Estructura de un Web Component Nativo

Los Web Components nativos se basan en la clase `HTMLElement`. Para declarar y registrar un componente, utilice la siguiente estructura:

```
class MiComponente extends HTMLElement {  
  constructor() {  
    super(); // Obligatorio al heredar  
    console.log("Componente instanciado");  
  }  
  
  connectedCallback() {
```

```
// Se ejecuta al agregarse al DOM
this.innerHTML = "<p>Hola Mundo</p>";
}
}

// Registro en el navegador (debe llevar un guion)
customElements.define("mi-componente", MiComponente);
```

2. Shadow DOM y Plantillas (Templates)

Para encapsular el HTML y el CSS de un componente, aislándolo de los estilos globales, se utiliza **Shadow DOM** junto con un elemento `<template>`:

```
const template = document.createElement("template");
template.innerHTML = `
  <style>
    p { color: var(--primary-color, teal); }
  </style>
  <div>
    <h3><slot name="titulo">Sin título</slot></h3>
    <p><slot name="contenido">Vacio</slot></p>
  </div>
`;

class TarjetaInfo extends HTMLElement {
  constructor() {
    super();
    // Adjuntar Shadow Root en modo abierto
    this.attachShadow({ mode: "open" });
    // Clonar y adjuntar plantilla
    this.shadowRoot.appendChild(
      template.content.cloneNode(true)
    );
  }
}
customElements.define("tarjeta-info", TarjetaInfo);
```

3. Manejo de Atributos Dinámicos y data-*

Para enviar parámetros desde el archivo HTML al componente y reaccionar a sus cambios en tiempo real, implemente el ciclo de vida de atributos:

```
class TarjetaLibro extends HTMLElement {
  // Declarar atributos observados
  static get observedAttributes() {
    return ["data-titulo", "data-autor"];
  }

  // Callback de cambio de atributos
  attributeChangedCallback(name, oldValue, newValue) {
    if (oldValue !== newValue) {
      this.render();
    }
  }

  connectedCallback() {
    this.render();
  }

  render() {
    // Leer atributos mediante la API dataset
    const titulo = this.dataset.titulo || "N/A";
    const autor = this.dataset.autor || "Anónimo";

    // Inyectar en el DOM interno o Shadow DOM
    if (this.shadowRoot) {
      // Lógica para actualizar elementos internos
    }
  }
}
```

4. Consumo de Servicios Web con Fetch API (Async/Await)

Para interactuar de forma asíncrona con los controladores de Spring Boot, utilice la Fetch API con funciones asíncronas (async/await):

Petición GET (Cargar datos)

```

async function cargarLibros() {
  try {
    const respuesta = await fetch("http://localhost:8080/api/libros");

    if (!respuesta.ok) {
      throw new Error(`HTTP error: ${respuesta.status}`);
    }

    const libros = await respuesta.json();
    const contenedor = document.querySelector(".grid");
    contenedor.innerHTML = ""; // Limpiar

    libros.forEach(libro => {
      const tarjeta = document.createElement("tarjeta-libro");
      // Setear atributos data-*
      tarjeta.setAttribute("data-titulo", libro.titulo);
      tarjeta.setAttribute("data-autor", libro.autor);

      contenedor.appendChild(tarjeta);
    });
  } catch (error) {
    console.error("Error al cargar libros:", error);
  }
}

```

Petición POST (Guardar datos con validación)

```

async function guardarLibro(nuevoLibro) {
  try {
    const respuesta = await fetch(
      "http://localhost:8080/api/libros",
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(nuevoLibro)
      }
    );

    if (respuesta.status === 201) {
      const libroCreado = await respuesta.json();
      console.log("Guardado con éxito:", libroCreado);
      return true;
    }
  }
}

```

```
    } else if (respuesta.status === 400) {  
        const mensaje = await respuesta.text();  
        alert(`Error de validación: ${mensaje}`);  
    } else {  
        console.error("Error inesperado en el servidor");  
    }  
} catch (error) {  
    console.error("Error de red:", error);  
}  
return false;  
}
```

■ Consejos de Implementación

- **Evite etiquetas auto-cerradas:** Los Custom Elements siempre requieren etiqueta de cierre. Use `<tarjeta-libro></tarjeta-libro>` en lugar de `<tarjeta-libro />`.
- **Encapsulación de estilos:** Recuerde que el CSS global de `style.css` no penetrará en el Shadow DOM de `<tarjeta-libro>`. Todos los estilos específicos de la tarjeta deben colocarse dentro de la etiqueta `<style>` interna del template en su código JS.
- **Prevenir envío de formularios:** Al enlazar el evento `submit` de un formulario, siempre invoque `event.preventDefault()` como primera línea para evitar la recarga automática del navegador.